

CultureQuest - Platformă mobilă pentru explorare urbană bazată pe proximitate și Federated Learning

1. Introducere și viziune

Majoritatea aplicațiilor de tip travel oferă liste de locații sau sugestii bazate pe popularitate globală, însă nu țin cont suficient de preferințele individuale, de contextul în timp real sau de principiile de protecție a datelor.

CultureQuest propune o abordare diferită: o **aplicație mobilă** care transformă orașul într-un spațiu interactiv, personalizat și dinamic. Utilizatorul nu mai primește doar recomandări, ci parcurge un **traseu cultural** generat inteligent, adaptat în funcție de interesele sale, de timpul disponibil și de poziția geografică actuală. Mai mult, aplicația utilizează tehnologii moderne precum **Federated Learning** și **sisteme de proximitate** pentru a oferi personalizare fără a compromite confidențialitatea datelor.

Viziunea **CultureQuest** este de a crea o **experiență de explorare urbană** care combină tehnologia, cultura și interacțiunea socială într-un mod sigur, inteligent și captivant.

2. Descrierea aplicației

2.1 Descriere scurtă (pentru Connect)

CultureQuest este o aplicație mobilă care oferă trasee culturale personalizate în funcție de interesele, locația și contextul utilizatorului. Prin **detectarea proximității** față de **obiective culturale**, aplicația declanșează conținut interactiv precum povești locale, provocări educaționale și misiuni gamificate. Personalizarea este realizată prin **Federated Learning**, modelele fiind antrenate local pe dispozitiv, fără colectarea datelor comportamentale brute. Astfel, aplicația combină **explorarea urbană inteligentă** cu **protecția confidențialității utilizatorului**.

2.2 Descriere generală

CultureQuest este o aplicație mobilă dezvoltată folosind **Flutter**, destinată utilizatorilor care doresc să exploreze orașe într-un mod interactiv și educațional. La baza funcționării sale stă generarea de **trasee culturale dinamice**, care se adaptează constant în funcție de comportamentul și preferințele utilizatorului.

Atunci când un utilizator deschide aplicația, acesta își poate selecta **interesele principale** (de exemplu: **artă, arhitectură, istorie, gastronomie**). Pe baza acestor

informații și a locației curente, aplicația generează un **traseu optimizat**. Pe măsură ce utilizatorul se deplasează prin oraș, sistemul detectează apropierea de obiective culturale relevante prin intermediul platformei **Matchmore** și declanșează conținut contextual.

Acest conținut poate include:

- povești locale asociate clădirii sau zonei
- informații istorice detaliate
- mini-provocări educaționale
- misiuni culturale
- note virtuale lăsate de alți utilizatori

Un element definitoriu al aplicației este utilizarea **Federated Learning** pentru personalizare. În loc să colecteze date comportamentale pe server, **aplicația antrenează local** un model de recomandare pe dispozitivul utilizatorului. Doar parametrii modelului sunt trimiși către server pentru agregare, fără transmiterea datelor brute.

Această abordare face ca **personalizarea** să fie **eficientă**, dar în același timp **sigură** și conformă cu reglementările privind protecția datelor.

3. Studiu de piață

3.1 Contextul pieței aplicațiilor de travel

Piața globală a aplicațiilor de travel este dominată de **câteva categorii principale**:

- aplicații de navigație și hărți
- aplicații de recenzii și recomandări
- aplicații de planificare turistică
- aplicații de experiențe și tururi ghidate
- aplicații de tip gamification bazate pe locație

Totuși, **foarte puține aplicații** combină:

- personalizare avansată
- declanșare contextuală în timp real

- **machine learning distribuit**
- protecție activă a datelor utilizatorilor

3.2 Analiza competitorilor principali

Google Maps

Google Maps oferă:

- navigație
- recomandări de locații
- sugestii bazate pe istoric
- popularitate și recenzii

Limitări relevante:

- personalizare centralizată
- lipsa unui traseu cultural structurat
- lipsa gamificării
- lipsa unui sistem educațional integrat

Google Maps este orientat **pe eficiență și navigație**, nu pe experiență culturală.

Tripadvisor

Tripadvisor oferă:

- recenzii detaliate
- clasamente
- recomandări populare

Limitări:

- experiență statică
- lipsa adaptării dinamice la context
- nu oferă trasee optimizate personal
- lipsa tehnologiilor ML distribuite

GetYourGuide / Airbnb Experiences

Aceste aplicații oferă experiențe organizate.

Limitări:

- experiențe predefinite
- lipsa personalizării dinamice
- dependență de organizatori umani
- nu oferă adaptare în timp real

Geocaching

O aplicație bazată pe explorare prin locație.

Limitări:

- orientată pe joc, nu pe educație culturală
- fără sistem avansat de recomandare
- fără personalizare ML

3.3 Aplicații care folosesc Federated Learning

În prezent, **Federated Learning** este utilizat în:

- tastaturi predictive (ex: sugestii text)
- aplicații de sănătate
- sisteme IoT
- asistenți vocali

În zona turismului și explorării urbane, **utilizarea FL este extrem de rară sau inexistentă**.

Aceasta creează o **oportunitate strategică** pentru CultureQuest.

3.4 Market gaps identificate

Pe baza analizei, se identifică următoarele lipsuri în piață:

- lipsa personalizării locale fără colectare centralizată

- lipsa traseelor culturale generate inteligent
- lipsa adaptării la context în timp real
- lipsa unui sistem care combină proximitate + ML + gamificare

CultureQuest adresează exact aceste goluri.

4. Arhitectura din punct de vedere tehnologic

4.1 Frontend - Flutter

Aplicația mobilă este dezvoltată utilizând Flutter. Alegerea acestei tehnologii este motivată de faptul că Flutter permite **dezvoltarea unei aplicații cross-platform** dintr-o singură bază de cod, reducând timpul și costurile de dezvoltare.

Comparativ cu alte framework-uri cross-platform, Flutter oferă **performanță apropiată de aplicațiile native** și un control mai bun asupra interfeței grafice. Pentru o aplicație orientată pe experiență vizuală și interactivă, acest aspect este esențial.

De asemenea, Flutter permite **integrarea cu TensorFlow Lite**, ceea ce facilitează rularea modelelor de machine learning direct pe dispozitiv.

4.2 Backend - FastAPI

Backend-ul aplicației este implementat în FastAPI, un framework modern bazat pe Python.

Alegerea FastAPI este justificată prin performanța ridicată și **integrarea naturală cu ecosistemul Python** pentru machine learning. Deoarece Federated Learning este implementat folosind **Flower** (scris în Python), utilizarea FastAPI simplifică integrarea și comunicarea dintre componente.

FastAPI oferă suport pentru programare asincronă, documentație automată și o structură clară a codului, ceea ce îl face potrivit pentru o aplicație scalabilă.

4.3 Baza de date - MongoDB

Pentru stocarea datelor aplicației este utilizat MongoDB, o **bază de date NoSQL** orientată pe documente.

Această alegere este motivată de flexibilitatea structurii datelor. Conținutul cultural, notele virtuale și quest-urile pot avea structuri variabile, iar **modelul document-based permite adaptarea ușoară** fără migrații complexe de schemă.

MongoDB este scalabil și potrivit pentru aplicații cu **volum mare de date semi-structurate**.

4.4 Cache și optimizare - Redis

Redis este utilizat pentru **caching** și **optimizarea performanței**. Deoarece aplicația funcționează în timp real și implică recomandări dinamice, este important ca anumite date să fie accesate rapid.

Redis, fiind o bază de date in-memory, oferă **latență foarte mică** și **reduce încărcarea** asupra MongoDB.

4.5 Sistem de proximitate - Matchmore

Matchmore este responsabil pentru **detectarea proximității** dintre utilizatori și obiective culturale. În loc să implementeze **manual** un **sistem complex de geofencing**, aplicația utilizează infrastructura optimizată a Matchmore pentru **matching în timp real**.

Această soluție permite declanșarea rapidă și eficientă a conținutului contextual atunci când utilizatorul se apropie de un obiectiv relevant.

4.6 Federated Learning - Flower

Pentru implementarea Federated Learning este utilizat Flower.

Alegerea **Flower în locul TensorFlow Federated** este justificată prin **flexibilitate** și **orientare spre producție**. TensorFlow Federated este mai potrivit pentru cercetare academică și simulări, în timp ce Flower permite implementarea mai ușoară a unui **sistem real distribuit**.

Flower suportă integrarea cu modele TensorFlow care pot fi convertite în **TensorFlow Lite** pentru **rulare pe dispozitive mobile**. În acest mod, modelul global este distribuit către dispozitive, antrenat local, iar actualizările sunt trimise către server pentru agregare.

Integrarea cu **FastAPI** este **directă**, deoarece ambele folosesc Python.

5. Federated Learning - Arhitectură și algoritmi

Federated Learning reprezintă componenta centrală de inovare a aplicației.

5.1 Arhitectura generală FL

Sistemul urmează modelul clasic **client-server**:

- Dispozitivele mobile → **clienți federated**
- Server FastAPI + Flower → **orchestrator**

Fluxul:

1. Serverul trimite modelul global inițial.
2. Modelul este convertit în TensorFlow Lite.
3. Fiecare dispozitiv:
 - antrenează modelul local pe datele proprii
 - calculează actualizarea parametrilor
4. Se trimit doar weights sau gradients.
5. Serverul agregă actualizările.
6. Modelul global este actualizat și redistribuit.

5.2 Algoritmi de Federated Learning posibili

5.2.1. Federated Averaging (FedAvg)

FedAvg reprezintă **algoritmul standard** de agregare în Federated Learning și va fi implementat în faza inițială a aplicației.

Mod de funcționare:

- Serverul trimite modelul global către un set de utilizatori selectați.
- Fiecare dispozitiv:
 - antrenează modelul local pentru câteva epoci folosind datele proprii;
 - actualizează parametrii modelului.
- Dispozitivele trimit către server doar parametrii actualizați (nu datele brute).
- Serverul calculează o **medie ponderată a parametrilor primiți**.
- Se obține astfel un nou model global, redistribuit către utilizatori.

Avantaje:

- algoritm simplu și ușor de implementat;
- eficiență ridicată în ceea ce privește comunicarea;
- scalabilitate bună pentru un număr mare de utilizatori;
- stabilitate în scenarii distribuite.

Relevanță pentru CultureQuest:

- permite antrenarea modelului de recomandare fără colectarea datelor comportamentale;
- funcționează bine pentru un **model de tip MLP**;
- este **complet compatibil** cu **Flower** și cu arhitectura **FastAPI**.

5.2.2. FedProx

FedProx este o **extensie** a FedAvg, concepută pentru situații în care **datele utilizatorilor** sunt **foarte diferite** (non-IID).

În contextul CultureQuest, utilizatorii pot avea:

- preferințe culturale foarte variate;
- comportamente diferite în orașe diferite;
- volume de date locale dezechilibrate.

Mod de funcționare:

- similar cu **FedAvg**, dar fiecare client este **“penalizat”** dacă se îndepărtează prea mult de **modelul global**;
- introduce un **mecanism de regularizare** care limitează divergența modelelor locale.

Avantaje:

- stabilizează procesul de antrenare;
- reduce riscul ca modelele locale să devină foarte diferite între ele;
- îmbunătățește convergența în condiții de eterogenitate ridicată.

Relevanță pentru aplicație:

- potrivit atunci când **preferințele** utilizatorilor sunt **puternic personalizate**;
- **recomandat pentru fazele ulterioare**, după MVP.

5.2.3. FedAdam / FedOpt

FedAdam și alte variante din familia **FedOpt** reprezintă **metode de optimizare adaptative** pentru Federated Learning, inspirate din algoritmi precum Adam utilizați în deep learning clasic.

Mod de funcționare:

- în loc să realizeze doar o medie simplă a parametrilor;
- serverul aplică o **actualizare adaptativă** asupra modelului global;
- ține cont de **istoricul actualizărilor** pentru a ajusta mai eficient **direcția de optimizare**.

Avantaje:

- convergență mai rapidă;
- performanță mai bună pe date eterogene;
- stabilitate crescută în sisteme cu mulți clienți.

Relevanță pentru CultureQuest:

- util atunci când **numărul utilizatorilor crește semnificativ**;
- potrivit pentru **optimizarea pe termen lung** a modelului contextual.

5.2.4. Secure Aggregation

Secure Aggregation nu este un algoritm de optimizare, ci un **mecanism de securitate** aplicat în procesul de agregare.

Principiu:

- serverul nu poate vedea actualizările individuale ale utilizatorilor;
- **doar rezultatul agregat final este accesibil**;
- fiecare actualizare este criptată înainte de a fi trimisă.

Avantaje:

- protecție suplimentară a confidențialității;
- reduce riscul expunerii actualizărilor individuale;
- aliniere puternică cu principiile **privacy-by-design**.

Relevanță pentru aplicație:

- **recomandat pentru versiuni viitoare**;
- consolidează poziționarea CultureQuest ca aplicație orientată spre **protecția datelor**.

5.3 Modelul de recomandare utilizat

Modelul de recomandare din CultureQuest are rolul de a estima relevanța fiecărui obiectiv cultural pentru un utilizator, într-un anumit context. Deoarece aplicația utilizează Federated Learning, modelul trebuie să fie:

- **eficient computațional** (rulare pe dispozitive mobile)
- compatibil cu TensorFlow Lite
- stabil în condiții de **date non-IID**
- suficient de expresiv pentru a modela preferințe diverse

Pentru versiunea inițială a aplicației se utilizează un **MLP (Multi-Layer Perceptron)**, extins ulterior într-o **variantă contextuală**.

5.3.1. Rețea neuronală simplă (MLP)

MLP-ul este un model **feed-forward** utilizat pentru a calcula un scor de relevanță pentru fiecare obiectiv cultural.

Input-ul modelului

Caracteristicile introduse în model pot include:

- categorii preferate (artă, istorie, gastronomie etc.)
- frecvența interacțiunilor cu fiecare categorie
- timp mediu petrecut la obiective similare
- număr de vizite anterioare
- tipul obiectivului (muzeu, monument, parc etc.)
- distanța față de utilizator

Aceste date sunt transformate într-un vector numeric și normalizate.

De ce MLP?

MLP este potrivit deoarece:

- modelează **relații non-liniare** între preferințe și caracteristici
- este **ușor de antrenat local**

- este eficient pe dispozitive mobile
- poate fi convertit în TensorFlow Lite
- este **compatibil cu algoritmi de Federated Learning** precum FedAvg

Pentru date tabulare și caracteristici numerice, MLP reprezintă un **echilibru** bun între **performanță** și **simplitate**.

5.3.2. Model contextual

Pentru a îmbunătăți relevanța recomandărilor, modelul poate fi extins prin includerea **variabilelor contextuale**.

Variabile contextuale posibile:

- locația curentă (sau zona urbană)
- ora din zi
- zi a săptămânii
- densitatea obiectivelor în proximitate
- timpul rămas disponibil

Aceste variabile sunt **concatenate cu vectorul de profil** al utilizatorului și introduse în MLP.

Beneficiile modelului contextual

Prin includerea contextului, sistemul poate:

- recomanda obiective diferite dimineața vs. seara
- prioritiza obiective apropiate când timpul este limitat
- adapta sugestiile în funcție de aglomerație sau densitate

Astfel, modelul nu învață doar „ce preferă utilizatorul”, ci „**ce este relevant** pentru utilizator **în situația actuală**”.

5.3.3. Strategie de implementare

Pentru CultureQuest, **abordarea graduală** este cea mai realistă:

- Faza 1: **MLP + FedAvg** (implementare inițială)

- Faza 2: Migrare către **FedProx** dacă se observă instabilitate din cauza datelor non-IID
- Faza 3: Optimizare cu **FedAdam** pentru **convergență** mai **rapidă**
- Faza 4: Integrare **Secure Aggregation** pentru **securitate avansată**

Această progresie permite dezvoltarea controlată a sistemului, fără a introduce complexitate inutilă în faza inițială.

6. Concluzie

CultureQuest este o aplicație inovatoare care combină:

- explorarea urbană
- proximitatea contextuală
- **machine learning distribuit**
- protecția avansată a datelor

Prin utilizarea **Flutter**, **FastAPI**, **MongoDB**, **Redis**, **Matchmore** și **Flower**, aplicația beneficiază de un stack modern, scalabil și coerent.

Componenta de **Federated Learning**, bazată pe algoritmi precum **FedAvg** și **FedProx**, reprezintă elementul de **inovație tehnologică majoră** și oferă un avantaj competitiv clar pe piața aplicațiilor de tip travel.